Worksheet Answers: RAG, Elasticsearch 101, and Agents

CS 5090 · Software Engineering with Generative AI · Week 7 · answers with short explanations, for your own review. Where a question is open-ended, the answer below is one good answer, not the only one.

Part 1 · Why RAG, and what it is

1.1 *A confident description of a function that does not exist.*
(b) lack of private or proprietary data. The model has never seen your repository, so it cannot know `sync_ledger()`; it produces a plausible answer anyway because output generation is probabilistic.

1.2 *The four components in order.*
1 Ingestion: load the authoritative data into a data source such as a vector database or search index. **2 Retrieval**: fetch the pieces relevant to the user's query. **3 Augmentation**: combine the retrieved text and the query into one prompt. **4 Generation**: the model answers from that augmented prompt.

1.3 *Prompt engineering, RAG, or fine-tuning?*
Monthly-changing policy documents: **RAG** (needs external knowledge, no change to the model; re-index instead of re-train). Strict JSON style with no outside knowledge: **prompt engineering** (low on both axes). An unseen in-house language with 50,000 examples: **fine-tuning** (the model itself has to change).

Part 2 · Elasticsearch 101

2.1 *The inverted index.*

term	documents
bug	D1, D3
login	D1, D2
page	D2, D3
fix	D1

2.2 *Queries on the index.*
`login AND bug` → **D1** (the documents in both lists). `page` → **D2, D3**. It is faster because the index is built once, in preprocessing; answering a query means looking up a few terms and combining short lists, not reading every document each time.

2.3 *Shard and node.*
A **shard** is one piece of an index's data (b); a **node** is one machine in the cluster (a). Nodes hold shards, which is how an index is spread across machines.

2.4 *RAG in practice.*
Elasticsearch returns the **relevant documents** for the query (full-text, semantic, or hybrid search). The language model gets those documents as context together with the question, and **generates the answer from them**.

Part 3 · Where RAG fails

3.1 *Match to failure points.*
Ranked 12th, only top 5 used → **FP2 Missed Top-Ranked Documents**. Answer is there but a contradicting sentence wins → **FP4 Extraction Failure**. Product never mentioned, model answers anyway → **FP1 Missing Content**. Asked for a table, got paragraphs → **FP5 Wrong Format**.

3.2 *DocPrompting.*
It helps most with code that uses a **new or unseen library**. The model cannot know those functions from training, so the retrieved documentation supplies the names, signatures, and descriptions that connect what the user wants to the code that does it.

Part 4 · Agent basics

4.1 *"Fix this bug."*

1 Inspect the files and error report, **2 Plan** a fix, **3 Act** by editing the code, **4 Check** by running the tests, **5 Adapt** if they fail. The **decision–action–feedback loop**: what happens after an action decides what the agent does next.

4.2 *ReAct labels.*

`run_tests(module="api")` is an **Action**. `2 failed: ...` is an **Observation** (what came back). "The failures are both in upload; I should look at upload.py first." is a **Thought**.

4.3 *Tool calls and MCP.*

F: the model only emits the request; your program (the client or harness) runs it. **T**: an MCP server exposes tools, prompts, and resources. **F**: MCP is one protocol, so any client can use any server; no per-assistant, per-tool integration.

4.4 *What Reflexion adds.*

A **memory of self-reflections**. After a failed attempt the agent writes down, in words, what went wrong; on the next attempt it reads that reflection first, so it avoids repeating the same mistake instead of starting from zero.

4.5 *Which step says "done"?*

The **Check** step, running the tests. Software has cheap, automatic verifiers (tests, compilers), so the loop can tell success from failure without a human. That is why coding agents work better than agents in domains where nothing checks the result.